Виртуализация и ее типы, контейнеризация, Docker

Лекция 09: Виртуализация

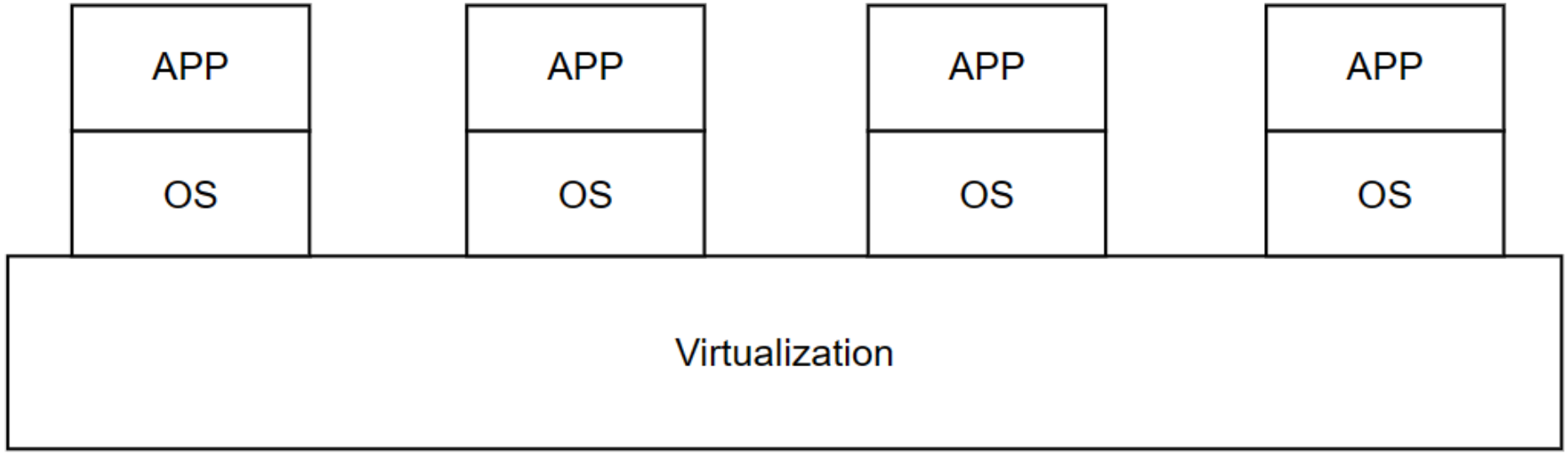
Материалы (контент) могут использоваться исключительно в учебных и информационных целях без цели извлечения прибыли с обязательным указанием следующей информации:
© ООО «КНС ГРУПП» (YADRO), 2022-2023, все права защищены.
Авторы – Александрова О. В., Андрианов А.А., Денисенко Б. А., Дутова А. А., Ефимов М. А., Клочкова Л. И., Королев Д. О., Котлярова Л. П., Никифоров И. В., Ромашов В. А., Тянутов М. В.

Содержание семинара

- Виртуализация
- Типы виртуализации
- Контейнеризация
- Docker
- Docker Compose

Виртуализация

Виртуализация — это процесс запуска виртуального экземпляра компьютерной системы на уровне, абстрагированном от реального оборудования.



Плюсы виртуализации

- Снижение затрат;
- Максимальная отдачу от инвестиций;
- Масштабируемость.

Минусы виртуализации

- Первоначальные затраты огромны;
- Не все оборудование или программное обеспечение можно виртуализировать;
- Легко увлечься добавлением серверов.

Типы виртуализации

- Виртуализация сети

Виртуализация сети позволяет создавать и настраивать виртуальные сети — логические коммутаторы, маршрутизаторы, брандмауэры, балансировщик нагрузки, виртуальную частную сеть (VPN) и обеспечивать безопасность рабочей нагрузки в течение нескольких дней или даже недель.

- Виртуализация рабочего стола

Виртуализация рабочих столов позволяет удаленно хранить ОС пользователей на сервере в центре обработки данных. Это позволяет пользователю получить доступ к своему рабочему столу виртуально из любого места с другого компьютера.

- Виртуализация систем хранения данных

Виртуализация хранилища — это массив серверов, которыми управляет виртуальная система хранения. Серверы не знают, где именно хранятся их данные, и вместо этого функционируют скорее как рабочие пчелы в улье. Это позволяет управлять хранилищем из нескольких источников и использовать его как единый репозиторий.

Типы виртуализации

- Виртуализация серверов

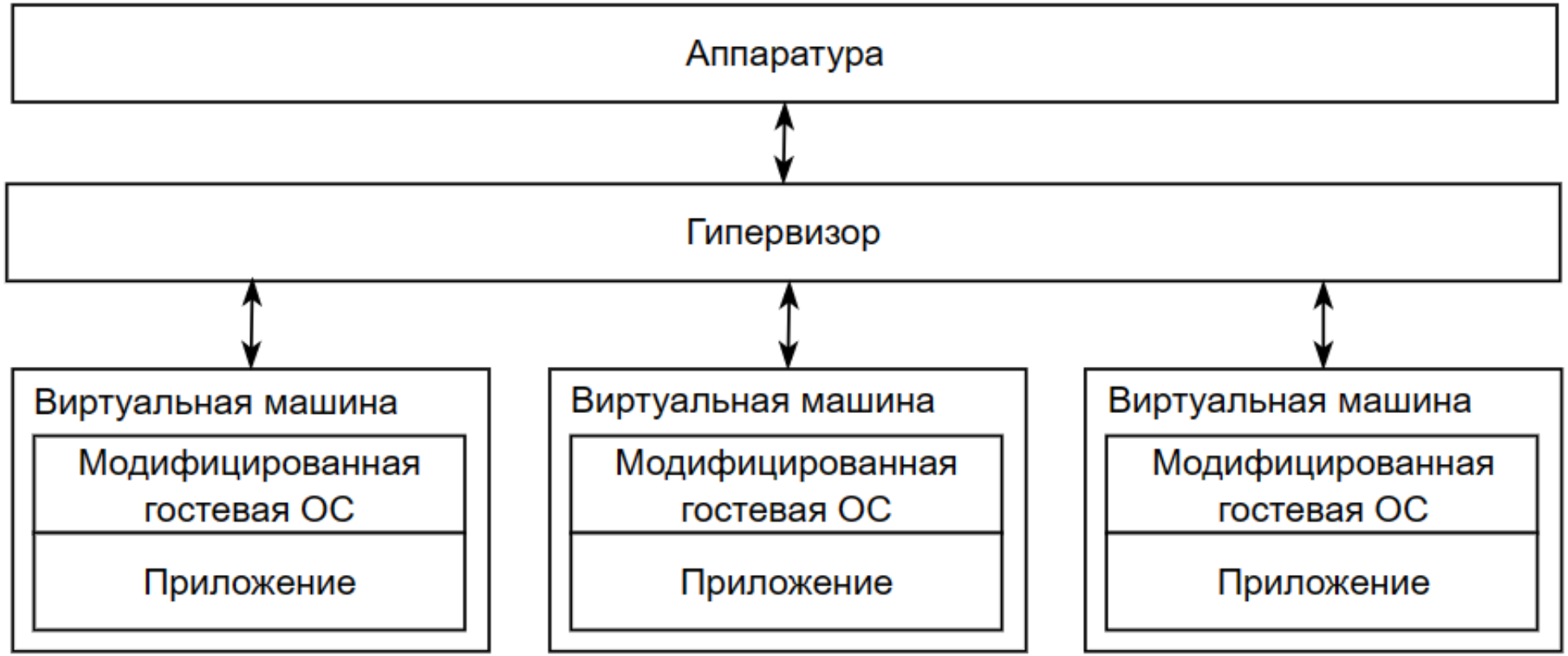
Центральный сервер (физический сервер) разделен на несколько разных виртуальных серверов путем изменения идентификационного номера, процессоров. Таким образом, каждая система может работать со своими собственными операционными системами изолированным образом.

- Виртуализация операционных систем

1. Программная виртуализация;
2. Аппаратная виртуализация;
3. Контейнеризация.

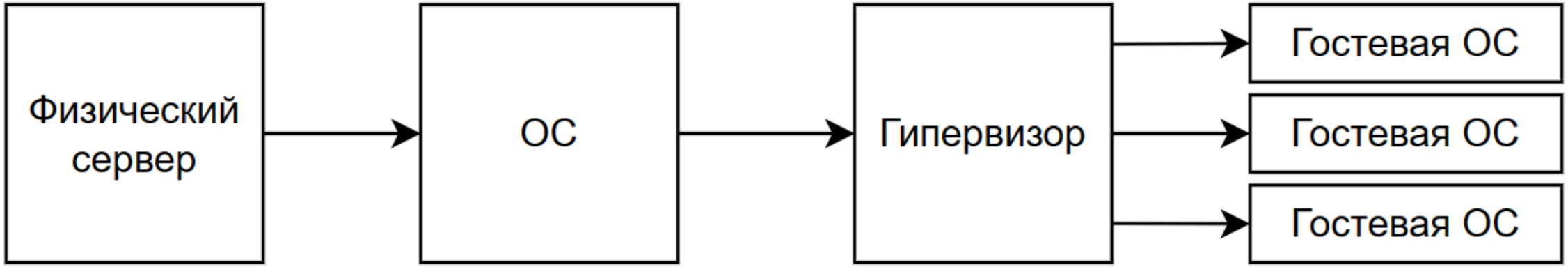
Аппаратная виртуализация

Подобная виртуализация осуществляется на основе процессорной архитектуры. При такой виртуализации с помощью гипервизора возможно прямое управление гостевыми системами, которые используются изолированно друг от друга.



Программная виртуализация

Программная виртуализация происходит на уровне ядра операционной системы.

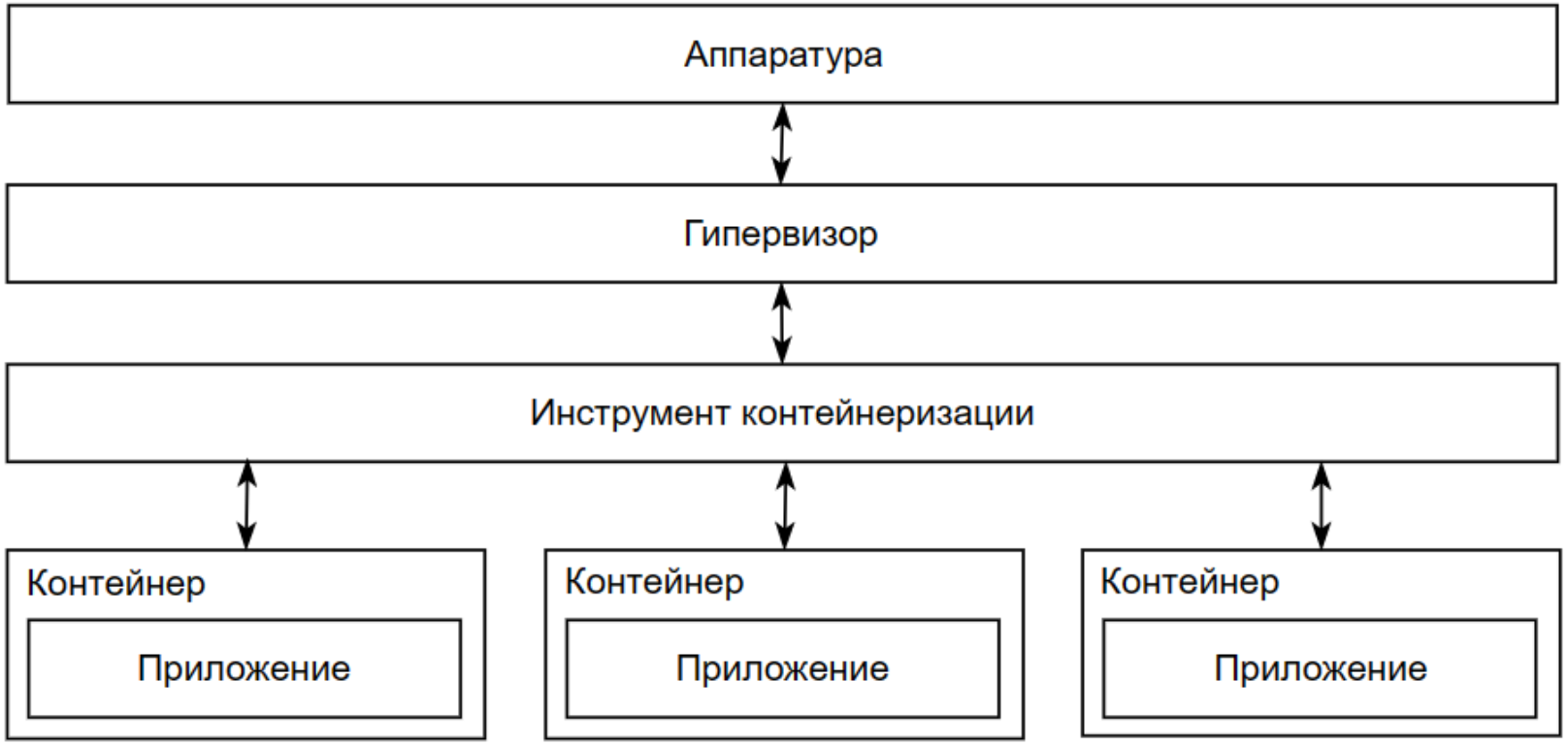


Типы гипервизоров

- Гипервизоры типа 1 запускаются непосредственно на сервере без операционной системы с прямым доступом к аппаратным ресурсам. Гипервизоры с открытым исходным кодом: KVM, Proxmox и Xen. Коммерческие гипервизоры: Red Hat Enterprise Virtualization (RHEV), Citrix XenServer, Hyper-V и VMware ESXi.
- Гипервизоры типа 2 работают как программы поверх ОС хост-компьютера. Они используются для абстрагирования гостевых операционных систем от ОС хоста. Гипервизоры с открытым исходным кодом: QEMU и VirtualBox. Коммерческие гипервизоры: Parallels Desktop, VMware Workstation Player и VMware Fusion.

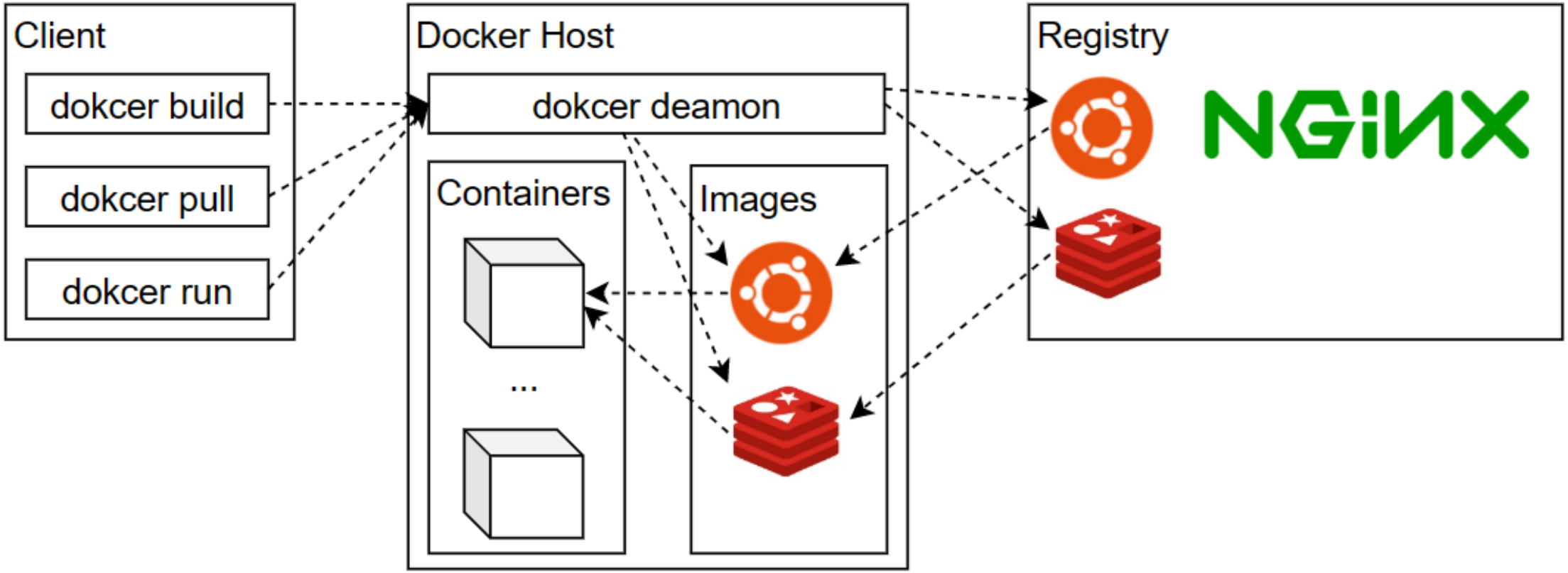
Контейнеризация

Контейнеризация - запуск программного обеспечения в изолированных пространствах на уровне операционной системы. Контейнер — это стандартная единица программного обеспечения, которая упаковывает код и все его зависимости, чтобы приложение быстро и надежно запускалось.



Docker

Docker – это программная платформа для быстрой разработки, тестирования и развертывания приложений.



DOCKERFILE

Для развертывания приложения необходимо создать Docker образ. Это делается с помощью Dockerfile.

```
FROM docker/whalesay:latest
RUN apt-get -y update && apt-get install -y
fortunes
CMD /usr/games/fortune -a | cowsay
```

docker-example.yaml

После того, как вы подготовили свой Dockerfile, вы можете использовать инструмент командной строки build для создания образа:

docker build –t docker-whale .

Docker. Команды

docker pull <image name>

Эта команда используется для извлечения образов из репозитория докеров (hub.docker.com).

docker run <image name>

Эта команда используется для создания контейнера из образа.

docker ps

Эта команда используется для вывода списка запущенных контейнеров.

docker ps -a

Эта команда используется для отображения всех запущенных и закрытых контейнеров.

Docker. Команды

docker exec -it <container id>

Эта команда используется для доступа к работающему контейнеру.

docker stop <container id>

Эта команда останавливает работающий контейнер

docker kill <container id>

Эта команда уничтожает контейнер, немедленно останавливая его выполнение.

docker commit <conatainer id> <username/imagename>

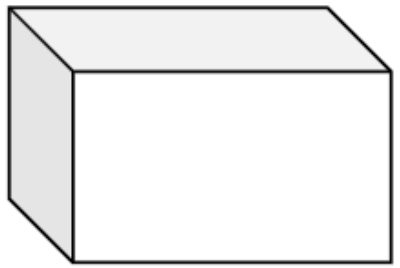
Эта команда создает новый образ редактируемого контейнера в локальной системе.

DockerHub

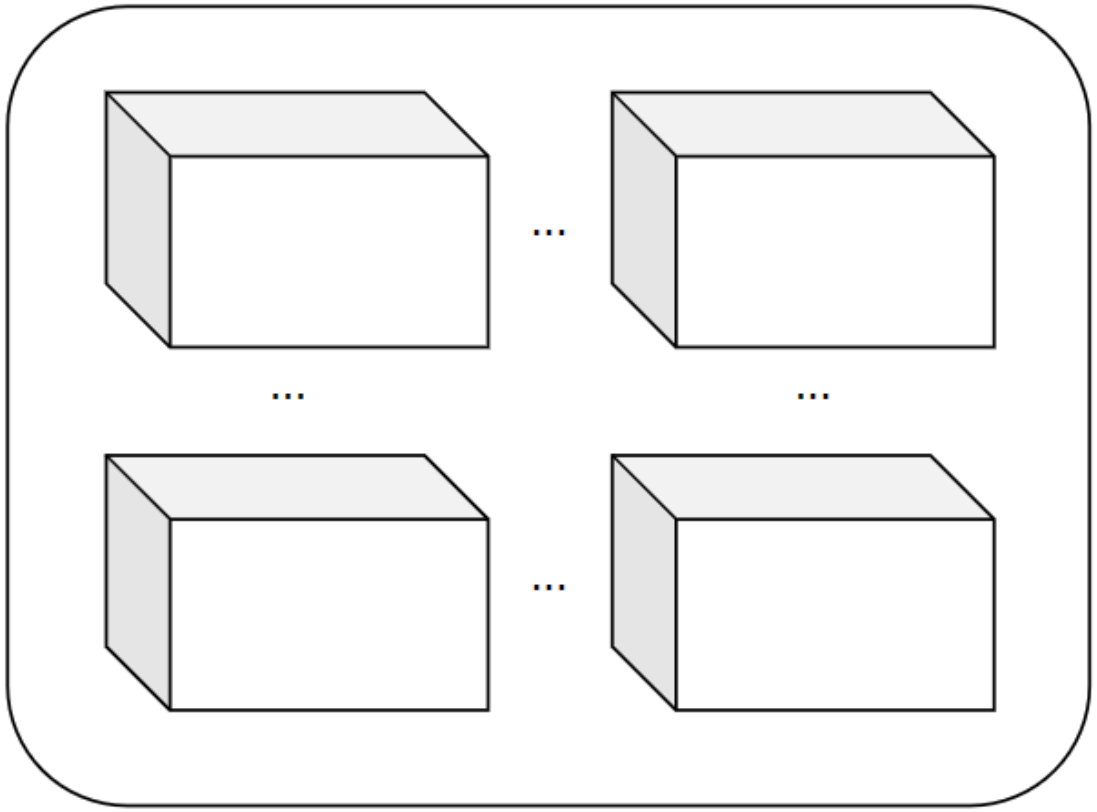
В профессиональных командах используются конвейеры CI/CD, в рамках которых происходит создание Docker образов, их загрузка в реестр контейнеров и дальнейшее развертывание с применением Docker на целевой платформе. Реестром контейнеров называется удаленный репозиторий, доступный из любой точки мира. Наиболее популярными реестрами контейнеров являются: Docker Hub и ECR (AWS Elastic Container Registry).

Docker Compose

Docker Compose — это инструмент, который позволяет легко запускать и обрабатывать несколько контейнеров одновременно.



Docker



Docker Compose

Docker Compose. Команды

docker-compose build

Собирает сервисы, описанные в конфигурационных файлах

docker-compose up

Запускает собранные сервисы

docker-compose up -d

Запуск контейнеров на фоне

docker-compose up --abort-on-container-exit

Если какой-то из сервисов завершит работу, то остальные будут остановлены автоматически

docker-compose down

Останавливает и удаляет все сервисы, которые были запущены с помощью up

Docker Compose. Конфигурация

Docker Compose работает, применяя множество правил, объявленных в одном файле конфигурации docker-compose.yml.

docker-compose up

В этом файле нам нужно указать версию формата файла Compose, хотя бы один сервис и, возможно, тома и сети.

```
version: "3.8"
services:
  ...
volumes:
  ...
networks:
  …
```

docker-compose-structure.yaml

Docker Compose. Конфигурация

Например, возьмем веб-приложение, состоящее из клиентской части, серверной части и базы данных. Разделим эти компоненты на три образа и определим их как три разных сервиса в конфигурации.

```
services:
  frontend:
    image: my-frontend-app
    ...
  backend:
    image: my-backend-app
    ...
  db:
    image: postgres
    …
```

docker-compose-service.yaml

Docker Compose. Конфигурация

Существует три типа томов: анонимный, именованный и хост. Docker управляет как анонимными, так и именованными томами, автоматически монтируя их в самостоятельно созданные каталоги на хосте.

```yaml
services:
  example-service:
    image: alpine:latest
    volumes:
      - my-volume:/volumes/volume
      - /tmp:/volumes/host-volume
      - /home:/volumes/readonly-host-volume:ro
```

docker-compose-volumes.yaml

Docker Compose. Конфигурация

Вместо того, чтобы просто использовать сеть приложения по умолчанию, вы можете указать свои собственные сети с помощью ключа сетей верхнего уровня. Это позволяет создавать более сложные топологии и задавать пользовательские сетевые драйверы и параметры.

```
networks:
  frontend:
    driver: driver-1
  backend:
    driver: driver-2
    driver_opts:
      foo: "1"
      bar: "2"
```

```
services:
  proxy:
    build: ./proxy
    networks:
      - frontend
  app:
    build: ./app
    networks:
      - frontend
      - backend
  db:
    image: postgres
    networks:
      - backend
```

docker-compose-network.yaml

yadro.com